IDEA SUBMISSION DOCUMENT · PS-01 · TEAM BOOTS N' GUTS

SENTINEL: GenAI-Powered Static Triage Pipeline

1. Executive Summary

Banks face a surge in fraudulent APKs bypassing traditional signature-based detection. Manual reverse engineering is too slow for active campaigns, and generic automated sandboxes produce unacceptable false positive rates on legitimate banking apps. **SENTINEL** is a targeted, automated static analysis pipeline. It combines rapid Python-based decompilation, targeted API artifact extraction, Retrieval-Augmented GenAI reasoning, and **Live C2 Infrastructure Mapping**. SENTINEL evaluates the *intent* of an APK without relying on file hashes, producing a compliance-ready forensic report with specific code-level evidence within 90 seconds.

The proposed architecture abandons flawed dynamic sandboxing in favor of deterministic static extraction. By strictly controlling the context window through code-slicing (e.g., isolating `SmsManager` calls), SENTINEL directly addresses the requirement for identifying evolving fraud patterns without suffering from AI hallucination or context degradation.

Core Technical Strategy: To achieve this within a rapid prototyping cycle, Team Boots N' Guts employs a strict API-contract development model. The extraction logic (Python/JADX) is entirely decoupled from the inference engine (LLM/Threat Intel), allowing asynchronous development and minimizing integration failure points on demo day. This ensures a highly reliable, compliant, and rapidly deployable solution for proactive fraud prevention.

2. Multi-Tiered Calibration Datasets

To ensure SENTINEL operates with high precision and near-zero false positive rates on legitimate banking applications, the inference engine is calibrated against a multi-tiered data matrix. We reject synthetic datasets in favor of wild-caught binaries and strict open-source baselines.

DATASET / SOURCE	FORMAT	STRATEGIC INTEGRATION PURPOSE
TIER 1: THE ACTIVE THREAT MATRIX (ZERO-DAY & BANKING TROJANS)		
MalwareBazaar (abuse.ch) Wild-Caught Binaries	Raw .apk files	Provides active, highly obfuscated Android banking trojans (e.g., Cerberus, Anubis, Hydra). Used to test SENTINEL's ability to extract SMS interception logic hidden beneath overlay attacks.
Koodous API Crowdsourced Intel	Raw .apk + YARA Rules	A collaborative platform for Android malware analysts. We use this to source specific malware families flagged by community YARA rules, validating that our GenAI detects the exact behaviors human analysts flag.
TIER 2: THE BENIGN CONTROL BASELINE (FALSE POSITIVE REDUCTION)		
AndroZoo Commercial Benign	Raw .apk files	A massive repository of apps with zero VirusTotal detections. We specifically source official Indian banking apps to ensure the model learns the difference between legitimate OTP auto-fill and malicious SMS theft.
F-Droid Repository Open-Source Baseline	Raw .apk + Source Code	The gold standard for clean code. Because F-Droid apps are strictly open-source and audited, they provide a mathematically clean "noise-free" baseline to calibrate the extraction engine against.
TIER 3: THREAT INTELLIGENCE & STRUCTURED ONTOLOGY		
MITRE ATT&CK for Mobile Adversary Tactics	JSON Matrix	Constrains the LLM. Instead of generating arbitrary risk narratives, the LLM maps extracted behaviors strictly to standardized technique IDs (e.g., `T1636.004: SMS Messages`).
OWASP MASVS Banking Security Standard	Framework	The Mobile Application Security Verification Standard. Used in the prompt engineering phase to force the LLM to evaluate the APK against banking-grade compliance requirements.

3. Solution Architecture

SENTINEL relies on a highly automated static extraction pipeline, isolating the GenAI's focus strictly to relevant threat vectors and actively hunting external command-and-control servers.

Phase 1: Automated Decompilation (The Scripting Layer)

Upon upload, a Python subprocess automatically unpacks the APK using `Apktool` and decompiles the DEX files using `JADX`. The pipeline parses the `AndroidManifest.xml` to extract permissions, exported components, and target SDKs.

Phase 2: Targeted Artifact Extraction (The Filter Layer)

Feeding an entire app to an LLM introduces hallucination risks. SENTINEL uses a Python extraction engine to scan the decompiled Java source for specific, high-risk API groupings (e.g., `SmsManager`, `HttpURLConnection`). When found, the engine extracts a localized 50-line code slice surrounding the call, along with hardcoded IPs and URLs.

Phase 3: Live C2 Infrastructure Mapping (Active Reconnaissance)

[Differentiator] Extracted hardcoded IPs and domains are not just listed; they are automatically piped into the **Shodan** / **VirusTotal APIs**. SENTINEL checks if the remote server is actively running known malware frameworks (e.g., Cobalt Strike) or hosted in high-risk ASNs, providing immediate threat context before the LLM even runs.

Phase 4: GenAI Reasoning Engine

The extracted code slices, manifest data, and live C2 threat intelligence are formatted into a strict JSON schema and passed to the LLM. A precise system prompt forces the AI to act as a reverse engineer, evaluating the intent of the code slices against the MITRE ATT&CK for Mobile framework.

Phase 5: Deterministic Risk Scoring & UI

The LLM outputs detected MITRE techniques. A deterministic Python engine calculates the final `0-100 Score`. The Streamlit dashboard renders the score, a global map of the C2 infrastructure, and the specific lines of decompiled Java code proving malicious intent.

4. Technical Stack

COMPONENT CATEGORY	TECHNOLOGIES USED
Static Extraction Pipeline	Python 3.10+, JADX-CLI, Apktool
Code Slicing & Parsing	Python `re` (Regex), Abstract Syntax Tree (AST) manipulation
Threat Intel & Recon	Shodan REST API, VirusTotal API v3
GenAI Inference Engine	Google Gemini 1.5 Flash / OpenAI GPT-4o-mini API via structured JSON Schema
Frontend & Reporting	Streamlit (Dashboard), ReportLab (Automated PDF Generation)